Pytorch basics

MLP

Introduction of Pytorch

- Pytorch? Open-source deep learning framework
- Implements tape-based auto differential methods, unlike Caffe, and TensorFlow

  - Computational graphs can be dynamically defined and executed,

    making it easy to create complex models

    · Static computational graphs: define a computational graph -> compile -> run it.

    Very efficient in product systems and mobile environments, but can be cumbersome in R&D

    · Dynamic computational graphs: support a more flexible, imperative style of development,

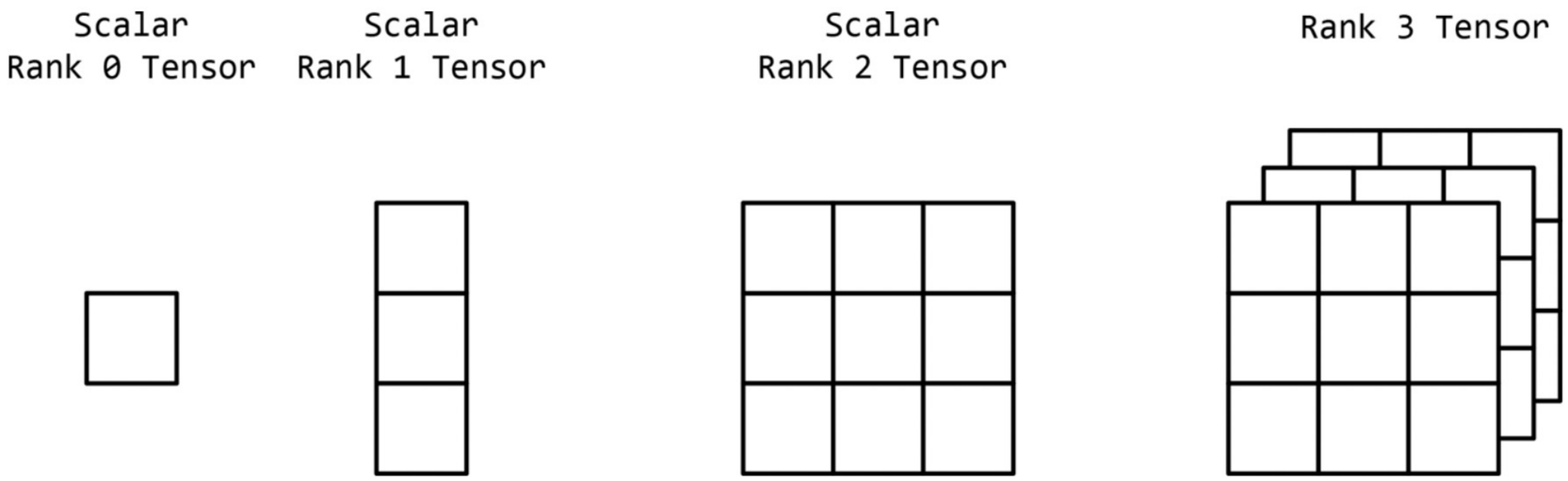
    and eliminate the need to compile the model each time you run it.

Introduction of Pytorch

- Tensor
  - Math objects with multidimensional data
  - A zero-order tensor is a single number or scalar,

    a first-order tensor is an array or vector of numbers,

    and a second-order tensor is an array or matrix of vectors.
  - Generalizable to n-dimensional scalar arrays



Introduction of Pytorch

- Creating a tensor

  - describe(x) : Helper functions to output properties of a tensor, such as tensor type, dimensions, and value.

```
def describe(x):
    print("타입: {}".format(x.type()))
    print("크기: {}".format(x.shape))
    print("값: \n{}".format(x))
```

  - Randomly initialize tensors by specifying dimensions

```
import torch
describe(torch.Tensor(2, 3))
```

executed in 7ms, finished 07:56:40 2023-02-01

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[2.9865e-34, 3.0875e-41, 2.9835e-34],
        [3.0875e-41, 1.1210e-43, 0.0000e+00]])
```

Introduction of Pytorch

- Creating a tensor

  - Generate a randomly initialized tensor with values sampled from a uniform distribution in the range [0,1] or a standard normal distribution.

```python
import torch
describe(torch.rand(2, 3))  # 균등 분포
describe(torch.randn(2, 3)) #표준 정규 분포
```

executed in 8ms, finished 08:07:07 2023-02-01

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[0.7507, 0.1176, 0.7844],
        [0.1210, 0.6067, 0.1371]])
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[-0.3764, -0.2229,  1.9274],
        [-0.6607,  1.2755,  1.8517]])
```

Introduction of Pytorch

- Creating a tensor
  - Create a tensor filled with a specific value using the fill()_ method
    - · An underscore character (_) are operations that change the value of a tensor.
    - · Similarly, there are in-place methods for normal distributions normal_(), uniform distributions uniform_(), etc.

```python
import torch
describe(torch.zeros(2, 3))
x = torch.ones(2, 3)
describe(x)
x.fill_(5)
describe(x)
```

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[0., 0., 0.],
        [0., 0., 0.]])
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[1., 1., 1.],
        [1., 1., 1.]])
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[5., 5., 5.],
        [5., 5., 5.]])
```

Introduction of Pytorch

- Creating a tensor

  - Create a tensor using a Python list

```
x = torch.Tensor([[1, 2, 3], [4, 5, 6]])
describe(x)
```

executed in 7ms, finished 08:22:03 2023-02-01

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

Create a tensor using an overflow array

· “DoubleTensor” is a default tensor type

→ Because the default data type for Numpy arrays is float64

```
import torch
import numpy as np
npy = np.random.rand(2, 3)
describe(torch.from_numpy(npy))
```

executed in 7ms, finished 08:28:11 2023-02-01

```
타입: torch.DoubleTensor
크기: torch.Size([2, 3])
값:
tensor([[0.4710, 0.3987, 0.4821],
        [0.9648, 0.4623, 0.4171]], dtype=torch.float64)
```

Introduction of Pytorch

- Tensor type and size
  - When using the torch.Tensor constructor, the default tensor type is FloatTensor
  - Tensor type can be specified at initialization or changed later with type casting
  - At initialization, you can specify the type by calling the constructor of a specific tensor type, such as FloatTensor or LongTensor, or by using the torch.tensor() method

```python
x = torch.FloatTensor([[1, 2, 3],
                       [4, 5, 6]])
describe(x)

x = x.long()
describe(x)

x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]], dtype=torch.int64)
describe(x)

x = x.float()
describe(x)
```

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[1., 2., 3.],
        [4., 5., 6.]])
타입: torch.LongTensor
크기: torch.Size([2, 3])
값:
tensor([[1, 2, 3],
        [4, 5, 6]])
타입: torch.LongTensor
크기: torch.Size([2, 3])
값:
tensor([[1, 2, 3],
        [4, 5, 6]])
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

Introduction of Pytorch

- Tensor Types and Sizes
  - You can determine the dimensions of a tensor using the shape property of the tensor object and the size() method.
  - When debugging Pytorch code, it's important to investigate tensor sizes.

```
x.size()
```

executed in 5ms, finished 08:41:13 2023-02-01

```
torch.Size([2, 3])
```

Introduction of Pytorch

- Tensor Operations

  - Perform operations using +, -, *, and / just like a typical programming language

  - You can also use functions like .add() instead of operators

```python
import torch
x = torch.randn(2, 3)
describe(x)

describe(torch.add(x, x))

describe(x + x)
```

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[ 0.5833,  0.2415,  0.3101],
        [ 0.6866, -0.8092,  0.2188]])
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[ 1.1666,  0.4831,  0.6202],
        [ 1.3731, -1.6184,  0.4376]])
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[ 1.1666,  0.4831,  0.6202],
        [ 1.3731, -1.6184,  0.4376]])
```

Introduction of Pytorch

- Tensor Operations

  - Operations that can be applied to specific dimensions of a tensor

```python
import torch
x = torch.arange(6)
describe(x)

x = x.view(2, 3)
describe(x)

describe(torch.sum(x, dim=0))

describe(torch.sum(x, dim=1))

describe(torch.transpose(x, 0, 1))
```

```
타입: torch.LongTensor
크기: torch.Size([6])
값:
tensor([0, 1, 2, 3, 4, 5])
타입: torch.LongTensor
크기: torch.Size([2, 3])
값:
tensor([[0, 1, 2],
        [3, 4, 5]])
타입: torch.LongTensor
크기: torch.Size([3])
값:
tensor([3, 5, 7])
타입: torch.LongTensor
크기: torch.Size([2])
값:
tensor([ 3, 12])
타입: torch.LongTensor
크기: torch.Size([3, 2])
값:
tensor([[0, 3],
        [1, 4],
        [2, 5]])
```

· arrange() : Create a tensor starts at 0 and increments by 1 up to a specified value

· view() : Create new tensors that share the same data

Introduction of Pytorch

- Indexing, slicing, and concatenation
  - Indexable, slicable, similar to Numpy

```python
import torch

x = torch.arange(6).view(2, 3)
describe(x)

describe(x[:1, :2])

describe(x[0, 1])
```

```
타입: torch.LongTensor
크기: torch.Size([2, 3])
값:
tensor([[0, 1, 2],
        [3, 4, 5]])
타입: torch.LongTensor
크기: torch.Size([1, 2])
값:
tensor([[0, 1]])
타입: torch.LongTensor
크기: torch.Size([])
값:
1
```

Introduction of Pytorch

- Indexing, slicing, and concatenation
  - See Complex Indexing, Non-Contiguous Tensor Indexes
    - The tensor returned by index_select() has the dimensions specified by dim equal to the length of the tensor specified by index.

      The remaining dimensions have the same size as the original tensor

```python
indices = torch.LongTensor([0, 2])
describe(torch.index_select(x, dim=1, index=indices))

indices = torch.LongTensor([0, 0])
describe(torch.index_select(x, dim=0, index=indices))

row_indices = torch.arange(2).long()
col_indices = torch.LongTensor([0, 1])
describe(x[row_indices, col_indices])
```

```
타입: torch.LongTensor
크기: torch.Size([2, 2])
값:
tensor([[0, 2],
        [3, 5]])
타입: torch.LongTensor
크기: torch.Size([2, 3])
값:
tensor([[0, 1, 2],
        [0, 1, 2]])
타입: torch.LongTensor
크기: torch.Size([2])
값:
tensor([0, 4])
```

Introduction of Pytorch

- Indexing, slicing, and concatenation
  - Connect tensors by specifying dimensions with built-in functions

```python
import torch

x = torch.arange(6).view(2,3)
describe(x)

describe(torch.cat([x, x], dim=0))

describe(torch.cat([x, x], dim=1))

describe(torch.stack([x, x]))
```

```
타입: torch.LongTensor
크기: torch.Size([2, 3])
값:
tensor([[0, 1, 2],
        [3, 4, 5]])
타입: torch.LongTensor
크기: torch.Size([4, 3])
값:
tensor([[0, 1, 2],
        [3, 4, 5],
        [0, 1, 2],
        [3, 4, 5]])
타입: torch.LongTensor
크기: torch.Size([2, 6])
값:
tensor([[0, 1, 2, 0, 1, 2],
        [3, 4, 5, 3, 4, 5]])
타입: torch.LongTensor
크기: torch.Size([2, 2, 3])
값:
tensor([[[0, 1, 2],
         [3, 4, 5]],

        [[0, 1, 2],
         [3, 4, 5]]])
```

Introduction of Pytorch

- 인덱싱, 슬라이싱, 연결

  - 파이토치는 텐서에서 매우 효율적인 선형 대수 연산 제공

    · 행렬 곱셈(mm), 역행렬(inverse, pinverse), 대각합(trace) 등

```python
import torch
x1 = torch.arange(6).view(2, 3).float()
describe(x1)

x2 = torch.ones(3, 2)
x2[:, 1] += 1
describe(x2)

describe(torch.mm(x1, x2))
```

```
타입: torch.FloatTensor
크기: torch.Size([2, 3])
값:
tensor([[0., 1., 2.],
        [3., 4., 5.]])
타입: torch.FloatTensor
크기: torch.Size([3, 2])
값:
tensor([[1., 2.],
        [1., 2.],
        [1., 2.]])
타입: torch.FloatTensor
크기: torch.Size([2, 2])
값:
tensor([[ 3.,  6.],
        [12., 24.]])
```

Introduction of Pytorch

- 텐서와 계산 그래프

  - 파이토치 tensor 클래스는 데이터와 대수연산, 인덱싱, 크기 변경 등 다양한 연산 캡슐화

    · 텐서의 requires_grad 매개변수를 True로 지정하면 그레이디언트 연산을 할 수 있는 텐서 생성

```python
import torch
x = torch.ones(2, 2, requires_grad=True)
describe(x)
print(x.grad is None)

y = (x + 2) * (x + 5) + 3
describe(y)
print(x.grad is None)

z = y.mean()
describe(z)
z.backward()
print(x.grad is None)
```

```
타입: torch.FloatTensor
크기: torch.Size([2, 2])
값:
tensor([[1., 1.],
        [1., 1.]], requires_grad=True)
True
타입: torch.FloatTensor
크기: torch.Size([2, 2])
값:
tensor([[21., 21.],
        [21., 21.]], grad_fn=<AddBackward0>)
True
타입: torch.FloatTensor
크기: torch.Size([])
값:
21.0
False
```

Introduction of Pytorch

- CUDA 텐서

  - GPU를 사용하려면 텐서를 GPU 메모리에 할당해야 함

  - CUDA API

    · NVIDIA 에서 만들었고, NVIDIA GPU에서만 사용할 수 있음

  - 내부적인 할당 방식만 다르고 CPU 텐서와 사용법이 같음

Introduction of Pytorch

- CUDA 텐서

  - torch.cuda.is_available()로 GPU를 사용할 수 있는지 확인

  - torch.device()로 장치 이름을 가져온뒤 .to(device)로 텐서를 타깃 장치로 이동

```python
import torch
print(torch.cuda.is_available())

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)

x = torch.rand(3, 3).to(device)
describe(x)
```

```
True
cuda
타입: torch.cuda.FloatTensor
크기: torch.Size([3, 3])
값:
tensor([[0.3753, 0.4631, 0.4456],
        [0.8755, 0.0060, 0.5917],
        [0.3439, 0.7928, 0.8748]], device='cuda:0')
```

Introduction of Pytorch

- CUDA 텐서

  - CUDA 객체와 CUDA가 아닌 객체를 다루려면 두 객체가 같은 장치에 있는지 확인해야 함

```
y = torch.rand(3, 3)
x + y
```

executed in 30ms, finished 09:52:04 2023-02-01

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Input In [41], in <cell line: 2>()
      1 y = torch.rand(3, 3)
----> 2 x + y

RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cuda:0 and
cpu!
```

```
cpu_device = torch.device("cpu")
y = y.to(cpu_device)
x = x.to(cpu_device)
x + y
```

executed in 16ms, finished 09:52:42 2023-02-01

```
tensor([[1.3692, 1.2164, 1.1527],
        [1.3015, 0.9541, 1.3979],
        [0.8240, 1.4793, 1.0815]])
```

  - GPU로 데이터를 넣고 꺼내는 작업은 비용이 많이 듦

    → 병렬 계산은 일반적으로 GPU에서 수행하고 최종 결과만 CPU로 정송하는 방식

  - CUDA 장치가 여럿이라면 CUDA_VISIBLE_DEVICES 환경변수를 사용

Introduction of Pytorch

- You'll need to understand the tutorial below to proceed with the class!
- https://pytorch.org/tutorials/beginner/pytorch_with_examples.html
- https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

감사합니다.